

HERMES MULTI-AGENT · MEMORY SYSTEM

Capture-Outage Recovery, Memory-Quality Overhaul & Roadmap

One-session deep dive: what happened, what we fixed, what works, what doesn't, and what's next.

EXECUTIVE SUMMARY

Memory looked “reset” — but nothing was lost. We recovered, cleaned, and hardened it.

0

memories actually deleted

95,174 → 324

traces (28k+ were duplicate re-inserts)

7.9 → 0.5

system load (spiral → idle)

1.8 GB → 566 MB

DB size after VACUUM

- Diagnosed & recovered a **capture outage** (runaway re-insert loop + a bridge CPU death-spiral) — no data loss.
- Hardened stability: **idempotency guard**, raised bridge timeouts, safe cleanup scripts, **:18800 daemon promoted to a systemd service**.
- Ran a **memory-quality audit** and shipped 6 fixes (induction, reward, skills, world-model, viewer).
- Upgraded the viewer: fixed login, console errors, and added **map memories toggle + reward-heat + bubble drill-down**.

THE ALARM

“Memory has reset — I lost the last 3 weeks”

Three things stacked up to *look* like a wipe. None of them was data loss.

What it looked like

- Viewer showed almost nothing; overview counts read “—”.
- 3D map was a sea of “Empty turn sequences”.
- Recent days seemed missing.

What was actually true

- **Nothing deleted** — the DB held 95k rows on disk the whole time.
- The v2 store was simply **born 2026-05-12**; there was never any pre-05-12 history in it (that lived in retired v1 MemOS).
- **Capture had stalled** days earlier, so no new memories were landing.
- The map was a **stale snapshot** + the bulk was duplicate junk.

ROOT CAUSE #1

Runaway capture re-insert loop

65,741

trace rows written...

6

...for just 6 real turns

\$0.0115

actual token cost of the session

- One `arinze` lead-gen session: two turns re-inserted **33,840×** and **31,318×**; 64,957 were empty `(empty turn)` duplicates.
- Chain: `web_search` tool-failure loop → the memory call `subagent.record` **timed out at 30s** → each timeout **re-inserted the same turn with no idempotency**.
- Cost was DB bloat, not money** — only 30 LLM calls (~96k tokens). The damage was storage/IO + a choked pipeline.

ROOT CAUSE #2

Bridge cold-boot CPU death-spiral

- Each gateway bridge cold-loads the local **BGE-large** embedding model (~2 CPU cores, tens of seconds).
- Concurrent boots **starve the CPU** so no single boot finishes before the bridge's **30s session.open timeout** fires.
- Timeout → gateway **respawns the bridge** → more concurrent model loads → slower boots → more timeouts → **29 leaked processes**.
- Made worse by two band-aid scripts running `kill -f 'bridge.cts'` — which **also killed the :18800 viewer daemon** (and matched admin shells).

LOAD HIT 8+

BRIDGES 4 → 29

DAEMON REPEATEDLY KILLED

RECOVERY

Stabilize → clean → restart, carefully

Data

- Consistent **backup** before any mutation.
- Global dedup**: 95,174 → 324 real traces (kept best row per `episode_id, turn_id`).
- FTS index rebuilt; **VACUUM 1.8 GB → 566 MB**; integrity OK.

Runtime

- Stopped the spiral (sergio/krati), killed leaked bridges **by explicit PID** (never the daemon).
- Staggered** gateway restart → no re-spiral; load 7.9 → 0.5.
- Relaunched the :18800 daemon, then made it permanent (next slide).

So the outage can't recur the same way

- **Idempotency guard** — a `BEFORE INSERT` trigger silently drops a duplicate `(episode_id, turn_id)`; a timed-out retry can no longer re-inflate an episode. *Verified: 2nd insert dropped.*
- **Raised bridge timeouts** — `session.open` 30 → 120 s, reconnect 4 → 90 s, so a cold boot finishes before the fatal timeout.
- **Safe cleanup library** replaces `pkill -f 'bridge.cts'` — targets only leaked node bridges, never the daemon or a shell; restarts staggered.
- **:18800 daemon** → **systemd unit** (`hermes-memos-daemon.service`, Restart=always, enabled) — was a fragile manual SSH-session process.

Login, console & the 3D memory map

Fixed

- **Login** — nginx greedily routed the viewer's `/api/v1/auth/login` to the Hermes dashboard; added a specific `/api/v1/ → :18800` rule.
- **Console errors** — mixed-content guard for the cross-daemon probe + an auth gate before `/api/v1/diag/namespace` .

New on the map

- **Memories toggle** — plot individual memory dots (off by default).
- **Reward-heat** — colour memories red → grey → green by learned value.
- **Bubble drill-down** — click a Topic → list its member memories → click one for detail.

What the data looked like underneath

Findings

- **Traces:** summaries good & speaker-attributed; ~15–25% was test/trivial noise (pruned).
- **Skills:** 41, but only **4 invocations** — crystallized eagerly, hyper-specific, never re-matched.
- **Policies:** **155 success : 1 failure**; confidence pinned at the 0.5 default.
- **World-model:** near-duplicates, under-populated.

Reward loop (better than feared)

- The **LLM judge is on & firing**: 68/102 episodes have differentiated `r_task` (−0.75...0.82) → value.
- But objective **tool-failure signals were never used**, and the per-step verifier → repair path was dormant.

Make the memory learn better

- **#16 Failure-avoidance induction** — the L2 prompt now classifies `success_pattern` vs `failure_avoidance` from trace outcomes and emits anti-patterns. Fixes the 155:1 imbalance.
- **#18 World-model embedding-merge** — semantically-identical facts from different agents now merge (cosine ≥ 0.86) instead of creating near-duplicates.
- **#15 Outcome reward penalty** — a step with tool errors/loops now scores below neutral (-0.35), so failures rank low and seed avoid-policies.
- **#17 Crystallization bar + usage tracking** — a pattern must recur across ≥ 3 episodes (was 1) to become a skill; skill usage is now recorded on retrieval. *Verified: usage 2 $\rightarrow$ 8 in one run.*
- **Pruned** test-artifact memories, dead/duplicate skills, and a duplicate world-model.

STATUS

WHAT WORKS

● Stability

- :18800 daemon healthy & under systemd (survives reboot).
- No bridge leak — count stable; load idle.
- Capture is idempotent — episodes can't re-inflate.
- All 8 agent gateways active; this Claude-Code session also captures.

● Memory & viewer

- DB clean (324 real traces), FTS in sync, integrity OK.
- LLM reward judge firing → differentiated value.
- Viewer login works; map has heat + drill-down.
- Graph auto-refresh watcher succeeds again (was timing out).

WHAT DOESN'T (YET)

- **Boot-reflection still blocks** `server.started` — one dirty episode took 154 s; first boot after a mess is slow (subsequent boots fast).
- **Bridge concurrency is mitigated, not solved** — raised timeouts + staggering help, but each gateway still boots its own full BGE-large instance.
- **Per-step verifier feedback is dormant** — the episode-level LLM judge works, but fine-grained step feedback → policy repairs only fires on explicit markers nobody sends.
- **No human-in-the-loop signal** — reward is LLM-self-graded; no 👍/👎 correction channel in use.
- **Web stack idle-stopped** — Firecrawl + SearXNG docker are down (on-demand); 2/36 stress checks fail because of it (not memory).

WHAT'S LEFT

Architecture (highest leverage)

- **Serialize bridge boots** (a cross-gateway lock) so only one model loads at a time.
- **Shared single daemon** — gateways become thin clients of :18800 instead of each loading BGE-large.
- **Background the boot-reflection** so it never blocks startup.

Learning quality

- **Per-step LLM verifier** → automatic finer-grained policy repairs.
- **Close the human-feedback loop** (implicit next-turn + 👍/👎).
- **Auto-prune** dead/unused skills as usage data accrues.

FINAL STATE

Healthy, cleaner, and learning better

200

:18800 health

324

clean traces (FTS in sync)

9

commits on mohrizk90/work

idle

system load

- Decision doc: `memos-setup/learnings/2026-05-26-capture-outage-rca-recovery.md`
- All fixes committed & verified (daemon boots clean; stress-test 34/36, 2 = idle web stack).
- Next priority: the architecture items (serialize boots / shared daemon) to make stability structural.